WIKIPEDIA

Retrieval Augmented Generation

Retrieval-Augmented Generation (**RAG**) is a software system that combines information retrieval with a large language model . A query that is sent to the system can access information from (external) information sources, databases or the World Wide Web instead of just the model's training data. [1] [2] This increases the accuracy and robustness of the generated content by providing the models with current and specific information. [3] Typical use cases are chatbots accessing internal (company) data or providing factual information that should come exclusively from reliable sources.

Table of contents

Proceedings
- data indexing
  - Chunking (splitting the data)
  - vector generation
  - networking of data
- data retrieval
  - filtering
  - rephrasing
  - Relational data
- augmentation
- response generation

applications

challenges

software packages

references

Proceedings

Retrieval-Augmented Generation essentially consists of four stages:

1. data indexing
2. data retrieval
3. augmentation
4. response generation

data indexing

In the first step, the data to be referenced is converted into word embeddings and indexed in a vector database . The data can be unstructured (text, images, video, audio), semi-structured, or structured (e.g. from a relational database or a knowledge graph [4]). In the vector database, the word embeddings are linked to the original data. [2]

Depending on the implementation, search queries to the system and the corresponding responses can also be indexed by a RAG system.

Chunking (splitting the data)

In the first step, the data to be indexed is divided into smaller components (*chunks*). Text documents, for example, can be divided into smaller paragraphs, while knowledge graphs can be divided into sub-graphs. Images can be divided according to the objects in the image (dog, table, face, etc.) using a segmentation model. [5] In relational databases, the database schema (e.g. DDL) can be divided into related tables that must be merged (e.g. "join").

The individual components are then annotated so that they can be assigned to the original data source.

vector generation

The individual chunks are broken down into tokens using tokenization . These tokens are then translated into word embeddings (vectors) using an embedding model. The word embeddings are then stored in the vector database together with the chunks and annotations.

In the case of image information, the image is translated into a description by an image-to-text model and this description is used for word embedding. The image itself is rarely encoded, as this requires a suitable tokenizer that can process image data. If image and text data are to be stored in the same database, a *multimodal tokenizer* is required. The same applies to processing audio and video data.

networking of data

Complex RAG systems have the ability to link data together automatically. A knowledge graph is extracted from the indexed documents using a language model and the concepts described within a document are linked to the respective concepts in the knowledge graph. [6] This link enables a formal analysis of the data that was previously only available in an unstructured form.

data retrieval

When retrieving data, a user 's search query is first converted into a word embedding. It is then determined which data stored in the vector database semantically correlates with the user's query . [7] [2] If relevant data is found, it is weighted (*ranking*) and filtered.

Different methods can be used to calculate the correlation. Typically, *Okapi BM25* [8] , SPLADE [9] , Dragon [10] , or a combination of several methods are used.

filtering

Data can be filtered for various reasons. Data confidentiality, child protection restrictions, copyrights and similar issues must be taken into account.

rephrasing

Some systems can evaluate search results for relevance criteria using a language model. If no relevant data is found, another agent can rephrase the user's query *and* start a new search. The system is limited in how many attempts it can make and/or how long it has to do so.

Since this method requires several additional steps, the system has to put in more work and time to deliver a result. The advantage is that there is a higher probability of obtaining a favorable search result.

Relational data

With relational data, the data is usually not completely converted into documents and stored in the vector database, but only the data schema. When searching, the necessary data schema is returned, from which a language model generates SQL source text (*Text-to-SQL*). This is executed on the database to generate the required information. [11]

It is important to note that the newly generated source code may be faulty or harmful. The query must therefore be checked for syntactical correctness (hallucination of non-existent tables or columns) as well as harmfulness (e.g. possible SQL injection). The database query must be "read-only" by setting appropriate permissions.

augmentation

In the augmentation process, application-specific data, such as a system prompt and the user's metadata , as well as the data found in the search query are merged with the user's query. [2] Complex RAG systems can divide the user's query into different knowledge areas and treat them specifically.

response generation

In the final step, the augmented search query is passed to a language model , which transforms the data into an answer using a statistical model [12] .

In addition, the response generated by the language model is linked to references to the sources of the data (typically via a URI) to enable the user to understand and control the response.

applications

Since RAG systems are an extension of classic search engines , the areas of application overlap. Applications for RAG systems can be found in customer service [13] [14] [15] , the analysis of medical information [16] [17] , legal research [18] , education [19] [20] , scientific research, financial analysis [21] [22] , the evaluation of log data [23] [24] , and all other applications where finding information is relevant.

challenges

RAG systems are highly complex to implement due to the large number of software systems and actors involved.

In addition, the indexing, weighting and filtering of the data must be continuously adapted and expanded to suit constantly changing circumstances. Operating a RAG system therefore not only involves a great deal of effort in implementation, but also a continuous effort in operation.

software packages

For the implementation of RAG systems, there are software packages that must be adapted to the specific application. A distinction is made between complex software frameworks , which enable extensive orchestration of AI agents and programming interfaces , and software packages that provide a low-code environment.

List of software packages for creating RAG systems (as of February 3, 2025)

software package	license	Description
Microsoft Copilot Studio (https://copilotstudio.microsoft.com/)	Proprietary	Low-code web platform in Microsoft 365
Azure AI Studio (https://azure.microsoft.com/en-us/products/ai-studio)	Proprietary	Low-code web platform in Microsoft Azure . Complex orchestration also possible.
PromptFlow (https://microsoft.github.io/promptflow/)	open source	Technology behind Copilot Studio and Azure AI Studio. Enables complex orchestration. Integration with LangChain or Semantic Kernel possible.
Vertex AI Agent Builder (https://cloud.google.com/products/agent-builder)	Proprietary	Google Cloud 's low-code web platform
watsonx (https://www.ibm.com/products/watsonx-ai)	Proprietary	low-code orchestration platform from IBM
Firecrawl (https://www.mendable.ai/?ref=gfirecrawl)	open source	API service for website indexing
LangChain (https://www.langchain.com/) and LangGraph (https://www.langchain.com/langgraph)	open source	Framework for orchestration of AI systems
Kernel Memory (https://github.com/microsoft/kernel-memory)	open source	Framework for creating RAG systems
Semantic Kernel (https://github.com/microsoft/semantic-kernel)	open source	Framework for orchestration of AI systems
Haystack (https://haystack.deepset.ai/)	open source	Framework for creating RAG systems
REALM (https://research.google/blog/realm-integrating-retrieval-into-language-representation-models/)	open source	Google Research project with a RAG system example implementation
RAGFlow (https://github.com/infiniflow/ragflow)	open source	A framework focused on simplifying the creation of RAG-based applications, with pre-built components and workflows.
txtai (https://github.com/neuml/txtai)	open source	An AI-powered data platform that offers search and question-answer capabilities and can be easily integrated into existing systems.
LlamaIndex (https://github.com/jerryjliu/llama_index)	open source	A tool for building indexes over your data to make it accessible to LLMs (Large Language Models).
RAGAS (https://github.com/explodinggradients/ragas)	open source	An evaluation framework for RAG systems that provides metrics to assess response quality.
RAGstack (https://github.com/gkamradt/ragstack)	open source	A stack for building RAG systems with components such as LangChain, FastAPI and others.
RAG Architect (https://github.com/hwchase17/rag-architect)	open source	A framework for creating RAG systems with a focus on modularity and extensibility.
RAG End-to-End (https://github.com/huggingface/transformers/tree/main/examples/research_projects/rag)	open source	An end-to-end example of implementing RAG with Hugging Face's Transformers.
RAG Pipeline (https://github.com/deepset-ai/haystack/tree/main/haystack/pipelines/rag)	open source	A pipeline within Haystack for implementing RAG systems.
STORM (https://github.com/stanford-oval/storm)	open source	An LLM-supported knowledge curation system developed by Stanford OVAL that creates Wikipedia-like articles based on Internet research.

software package	license	Description
LLM app (https://github.com/pathwaycom/llm-app)	open source	A framework from Pathway that makes it easier to build applications with large language models.
Cognita (https://github.com/truefoundry/cognita)	open source	A tool developed by TrueFoundry to integrate knowledge bases into AI-powered applications.
R2R (https://github.com/SciPhi-AI/r2r)	open source	A framework developed by SciPhi-AI for implementing retrieval-to-response systems.
neurites (https://github.com/satellitecomponent/neurite)	open source	A neural information processing and extraction tool developed by SatelliteComponent.
FlashRAG (https://github.com/RUC-NLPIR/flashrag)	open source	A framework for rapid retrieval-augmented generation developed by RUC-NLPIR.
canopy (https://github.com/pinecone-io/canopy)	open source	A tool developed by Pinecone to manage and orchestrate vector databases for RAG systems.

In addition, there are also ready-made software packages such as Pieces for Developers, which can be used for personal data management. [25]

Retrieval Augmented Generation is also used in some search engines such as Perplexity.ai , Google Gemini , Microsoft Bing , and SearchGPT.

references

1. Christopher D. Manning, Prabhakar Raghavan, Hinrich Schütze: *Introduction to Information Retrieval* . Ed.: Stanford University , Ludwig-Maximilians-University Munich . 2009, ISBN 978-0-521-86571-5 (English, stanford.edu (https://nlp.stanford.edu/IR-book/information-retrieval-book.html)).
2. Trey Grainger, Doug Turnbull, Max Irwin: *AI-Powered Search* . Ed.: Manning. 2024, ISBN 978-1-61729-697-0 (English, manning.com (https://www.manning.com/books/ai-powered-search) [accessed on July 23, 2024]).
3. Penghao Zhao et al.: *Retrieval-augmented generation for ai-generated content: A survey* . June 21, 2024, doi : 10.48550/arXiv.2402.19473 (https://doi.org/10.48550/arXiv.2402.19473) , arxiv : 2402.19473 (https://arxiv.org/abs/2402.19473) (English).
4. Tomaž Bratanič, Oskar Hane: *Knowledge Graph-Enhanced RAG* . Ed.: Manning. 2024, ISBN 978-1-63343-626-8 (English, manning.com (https://www.manning.com/books/knowledge-graph-enhanced-rag) [accessed on July 23, 2024]).
5. Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, Ross Girshick: *Segment Anything* . April 5, 2023, doi : 10.48550/arXiv.2304.02643 (https://doi.org/10.48550/arXiv.2304.02643) , arxiv : 2304.02643 (https://arxiv.org/abs/2304.02643) (English).
6. Junde Wu, Jiayuan Zhu, Yunli Qi: *Medical Graph RAG: Towards Safe Medical Large Language Model via Graph Retrieval-Augmented Generation* . August 17, 2024, doi : 10.48550/arXiv.2408.04187 (https://doi.org/10.48550/arXiv.2408.04187) , arxiv : 2408.04187 (https://arxiv.org/abs/2408.04187) (English).
7. Olesya Bondarenko: *Implement Semantic Search with ML and BERT* . Ed.: Manning. (English, manning.com (https://www.manning.com/liveproject/implement-semantic-search-with-ml-and-bert) [accessed July 23, 2024]).
8. Stephen E. Robertson, Hugo Zaragoza: *The Probabilistic Relevance Framework: BM25 and Beyond* . January 2009, doi : 10.1561/1500000019 (https://doi.org/10.1561/1500000019)

(English, researchgate.net (https://www.researchgate.net/publication/220613776_The_Probabilistic_Relevance_Framework_BM25_and_Beyond) [PDF; 1.3 MB ; accessed on July 23, 2024]).

9. *SPLADE.* (https://github.com/naver/splade) In: *GitHub* . Retrieved on August 17, 2024 (English).
10. Ayush Thakur: *llmware/dragon-mistral-7b-v0: A Powerful LLM for RAG Applications.* (https://blog.stackademic.com/llmware-dragon-mistral-7b-v0-a-powerful-llm-for-rag-applications-760fc6add4b9) Medium, December 22, 2023, accessed August 16, 2024 (English).
11. Pinterest Engineering: *How we built Text-to-SQL at Pinterest.* (https://medium.com/pinterest-engineering/how-we-built-text-to-sql-at-pinterest-30bad30dabff) Medium, April 2, 2024, accessed August 17, 2024 (English).
12. Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, Haofen Wang: *Retrieval-Augmented Generation for Large Language Models: A Survey* . Tongji University , Fudan University , March 27, 2024, arxiv : 2312.10997 (https://arxiv.org/abs/2312.10997) (English).
13. Keshav Unni: *Better Customer Support Using Retrieval-Augmented Generation (RAG) at Thomson Reuters.* (https://medium.com/tr-labs-ml-engineering-blog/better-customer-support-using-retrieval-augmented-generation-rag-at-thomson-reuters-4d140a6044c3) Medium , August 8, 2023, accessed August 18, 2024 (English).
14. Tarunvel VS: *RAG in Customer Support.* (https://medium.com/@TarunvelVS/rag-in-customer-support-aa971ff27ea8) Medium , April 11, 2024, accessed August 18, 2024 (English).
15. Adarsh: *Enhancing Customer Service with Retrieval-Augmented Generation (RAG) in AI Chatbots.* (https://www.kommunicate.io/blog/rag-in-customer-service-chatbot/) kommunicate.io (https://www.kommunicate.io/) , August 6, 2024, accessed August 18, 2024 (English).
16. Tanya Malhotra: *MedGraphRAG: An AI Framework for Improving the Performance of LLMs in the Medical Field through Graph Retrieval Augmented Generation (RAG).* (https://www.marktechpost.com/2024/08/12/medgraphrag-an-ai-framework-for-improving-the-performance-of-llms-in-the-medical-field-through-graph-retrieval-augmented-generation-rag/) August 12, 2024, accessed on August 16, 2024 (English).
17. Junde Wu, Jiayuan Zhu, Yunli Qi: *Medical Graph RAG: Towards Safe Medical Large Language Model via Graph Retrieval-Augmented Generation* . August 8, 2024, doi : 10.48550/arXiv.2408.04187 (https://doi.org/10.48550/arXiv.2408.04187) , arxiv : 2408.04187 (https://arxiv.org/abs/2408.04187) .
18. Arunim Samat: *Legal-RAG vs. RAG: A Technical Exploration of Retrieval Systems.* (https://www.truelaw.ai/blog/legal-rag-vs-rag-a-technical-exploration-of-retrieval-systems) truelaw.ai (https://www.truelaw.ai/) , March 23, 2024, accessed August 16, 2024 (English).
19. *Enhancing Education with RAG: How Universities Can Benefit.* (https://www.elementx.ai/post/enhancing-education-with-rag) ElementX.ai (https://www.elementx.ai/) , accessed August 16, 2024 (English).
20. *Unveiling the Impact of RAG AI in Education: Real Case Studies Revealed.* (https://myscale.com/blog/rag-ai-education-impact-case-studies/) MyScale (https://myscale.com/) , March 17, 2024, accessed August 16, 2024 (English).
21. Eduardo Alvarez: *Transforming Financial Services with RAG: Personalized Financial Advice.* (https://eduand-alvarez.medium.com/transforming-financial-services-with-rag-personalized-financial-advice-f58de9d2d7e0) Medium , April 5, 2024, accessed August 16, 2024 (English).
22. *Embracing RAG in Financial Services: A New Frontier in AI Technologies.* (https://www.linkedin.com/pulse/embracing-rag-financial-services-new-frontier-ai-technologies-nzfme/) In: *LinkedIn Pulse.* RoSoft Ventures, November 11, 2023, accessed August 16, 2024 (English).
23. Harikiran Akella: *Log Analysis with GenAI — With HandsOn.* (https://medium.com/@hari10.nitw/log-analysis-with-genai-with-handson-0e05224aed96) Medium , March 21, 2024, accessed August 16, 2024 (English).
24. Yi Xiao, Van-Hoang Le, Hongyu Zhang: *Stronger, Cheaper and Demonstration-Free Log Parsing with LLMs* . June 12, 2024, doi : 10.48550/arXiv.2406.06156 (https://doi.org/10.48550/arXiv.2406.06156) , arxiv : 2406.06156 (https://arxiv.org/abs/2406.06156) (English).

25. *Pieces.* (https://pieces.app/) Retrieved September 1, 2024 .